PROJECT BASED EXPERIMENTAL LEARNING

COURSE- DATABASE MANAGEMENT SYSTEM (NoSQL / MongoDB)

PROJECT REPORT

TOPIC- MONGODB STUDENT DATABASE OPERATIONS

IBM

SUBMITTED TO- MR. Harendra Singh Rajpoot
Submitted by- Aarsh
BCADS27
Roll no. - 1250258003
3rd semester
Babu Banarasi Das University
Lucknow, U.P

ACKNOWLEDGMENT

I would like to express my special thanks of gratitude to my mentor Mr. Harendra Singh Rajpoot, who gave me the golden opportunity to do this wonderful project on MongoDB Student Database Operations. My mentor helped me a lot in completing this project within the given time frame, and I learned many new things about NoSQL databases from him. He converted his experience into our knowledge.

Secondly, I want to show my gratitude to IBM, which provided me this project-based learning experience; it helped me to achieve my goals more efficiently.

Abstract

This report presents a MongoDB-based Student Database Management project that demonstrates the core Create, Read, Update and Delete (CRUD) operations of a NoSQL document database. A "students" collection is used to store student records containing roll number, name, age, department and marks. The project implements single and bulk insertion, conditional updates, single and bulk deletion, and a wide range of query operations using MongoDB comparison and logical operators such as $gt, $lt, $eq, $gte, $ne, $and, $or and $not. The implementation demonstrates how MongoDB can be used to efficiently manage, filter and retrieve structured student data.

Introduction

MongoDB is a popular NoSQL, document-oriented database that stores data as flexible, JSON-like documents rather than in rigid rows and columns. This makes it well suited for applications where the data structure may evolve over time, such as academic record-keeping systems. This project focuses on building and querying a "students" collection that stores academic details of 51 students, and demonstrates the standard database operations - insertion, updation, deletion and querying that form the backbone of any real-world database-driven application.

Methodology

The project was carried out step by step inside the MongoDB shell (mongosh), with every operation executed against a single collection named "students" in a database created for this purpose. The complete methodology, along with the exact code executed and the output produced by MongoDB at each step, is given below.

1. Insert One

Inserts a single student document into the students collection.

Code:

```
db.students.insertOne(
{ rollNo: 101, name:
"Vishnu", age: 22,
department: "CSE",
marks: 85 })
```

2. Insert Many

Inserts multiple student documents (roll numbers 102 to 151) into the collection at the same time.

Code:

```
db.students.insertMany([ { rollNo: 102, name: "Saraswati", age: 21,
department: "IT", marks: 78 },
{ rollNo: 103, name: "Govind", age: 20, department: "CSE", marks: 92 },
{ rollNo: 104, name: "Parvati", age: 19, department: "ECE", marks: 81 },
{ rollNo: 105, name: "Hari", age: 22, department: "ME", marks: 74 },
{ rollNo: 106, name: "Radha", age: 21, department: "CSE", marks: 88 },
{ rollNo: 107, name: "Mohan", age: 20, department: "IT", marks: 69 },
{ rollNo: 108, name: "Kamala", age: 19, department: "ECE", marks: 95 },
{ rollNo: 109, name: "Shankar", age: 22, department: "CSE", marks: 83 },
{ rollNo: 110, name: "Yamuna", age: 21, department: "IT", marks: 76 },
{ rollNo: 111, name: "Krishna", age: 20, department: "ME", marks: 72 },
{ rollNo: 112, name: "Sita", age: 19, department: "CSE", marks: 89 },
{ rollNo: 113, name: "Rajendra", age: 22, department: "ECE", marks: 67 },
{ rollNo: 114, name: "Kaveri", age: 21, department: "IT", marks: 91 },
{ rollNo: 115, name: "Gopal", age: 20, department: "CSE", marks: 79 },
{ rollNo: 116, name: "Savitri", age: 19, department: "ECE", marks: 84 },
{ rollNo: 117, name: "Bhairav", age: 22, department: "ME", marks: 73 },
{ rollNo: 118, name: "Rukmini", age: 21, department: "CSE", marks: 96 },
{ rollNo: 119, name: "Indra", age: 20, department: "IT", marks: 68 },
{ rollNo: 120, name: "Chandra", age: 19, department: "ECE", marks: 87 },
{ rollNo: 121, name: "Manohar", age: 22, department: "CSE", marks: 75 },
{ rollNo: 122, name: "Gauri", age: 21, department: "IT", marks: 90 },
{ rollNo: 123, name: "Vasudev", age: 20, department: "ME", marks: 71 },
{ rollNo: 124, name: "Uma", age: 19, department: "CSE", marks: 86 },
{ rollNo: 125, name: "Dwarka", age: 22, department: "ECE", marks: 80 },
{ rollNo: 126, name: "Kunti", age: 21, department: "IT", marks: 94 },
{ rollNo: 127, name: "Bhupendra", age: 20, department: "CSE", marks: 77 },
{ rollNo: 128, name: "Draupadi", age: 19, department: "ECE", marks: 82 },
{ rollNo: 129, name: "Ramesh", age: 22, department: "ME", marks: 65 },
{ rollNo: 130, name: "Lakshmi", age: 21, department: "CSE", marks: 93 },
{ rollNo: 131, name: "Suresh", age: 20, department: "IT", marks: 70 },
{ rollNo: 132, name: "Kausalya", age: 19, department: "ECE", marks: 89 },
{ rollNo: 133, name: "Prithvi", age: 22, department: "CSE", marks: 81 },
{ rollNo: 134, name: "Sarala", age: 21, department: "IT", marks: 85 },
{ rollNo: 135, name: "Mahesh", age: 20, department: "ME", marks: 78 },
{ rollNo: 136, name: "Janaki", age: 19, department: "CSE", marks: 97 },
{ rollNo: 137, name: "Narayan", age: 22, department: "ECE", marks: 74 },
{ rollNo: 138, name: "Urmila", age: 21, department: "IT", marks: 88 },
{ rollNo: 139, name: "Devendra", age: 20, department: "CSE", marks: 66 },
{ rollNo: 140, name: "Mandakini", age: 19, department: "ECE", marks: 92 },
{ rollNo: 141, name: "Ishwar", age: 22, department: "ME", marks: 69 },
{ rollNo: 142, name: "Shakuntala", age: 21, department: "CSE", marks: 84 },
{ rollNo: 143, name: "Trilok", age: 20, department: "IT", marks: 79 },
{ rollNo: 144, name: "Kamalini", age: 19, department: "ECE", marks: 91 },
{ rollNo: 145, name: "Ranjit", age: 22, department: "CSE", marks: 87 },
{ rollNo: 146, name: "Vidyawati", age: 21, department: "IT", marks: 73 },
{ rollNo: 147, name: "Chaturbhuj", age: 20, department: "ECE", marks: 86 },
{ rollNo: 148, name: "Sushila", age: 19, department: "ME", marks: 68 },
{ rollNo: 149, name: "Omkar", age: 22, department: "CSE", marks: 90 },
{ rollNo: 150, name: "Tulsi", age: 21, department: "IT", marks: 76 },
{ rollNo: 151, name: "Balram", age: 20, department: "ECE", marks: 83 }])
```

3. Update One

Updates the first document that matches the given condition - here, the marks of roll number 101 are updated from 85 to 90.

Code:

```
db.students.updateOne( { rollNo:
101 },
{ $set: { marks: 90 } }
)
```

Output:

```
{ acknowledged:
true,
matchedCount: 1,
modifiedCount: 1 }
```

4. Update Many

Updates all documents that match the given condition - here, every student in the CSE department is given a new status field set to "Active".

Code:

```
db.students.updateMany( { department:
"CSE" },
{ $set: { status: "Active" } }
)
```

Output:

```
{ acknowledged:
true,
matchedCount: 17,
modifiedCount:
17 }
```

5. Delete One

Deletes the first document that matches the given condition - here, the record of roll number 151 (Balram) is removed.

Code: `db.students.deleteOne({ rollNo: 151 })`

Output:

```
{ acknowledged:
true,
deletedCount: 1 }
```

6. Delete Many

Deletes all documents that match the given condition - here, every student with marks less than 40 is removed.

Code: `db.students.deleteMany({ marks: { $lt: 40 } })`

Output:

```
{ acknowledged:
true,
deletedCount: 0 }
```

7. Query Operations

A series of find() queries were run using MongoDB's comparison and logical operators to filter and retrieve student records.

Marks greater than 80:

```
db.students.find({ marks: { $gt:
80 } })
```

Output:

```
101 Vishnu 22 CSE 90
103 Govind 20 CSE 92
104 Parvati 19 ECE 81
106 Radha 21 CSE 88 108 Kamala 19 ECE 95
109 Shankar 22 CSE 83
112 Sita 19 CSE 89
114 Kaveri 21 IT 91
116 Savitri 19 ECE 84
118 Rukmini 21 CSE 96
120 Chandra 19 ECE 87
122 Gauri 21 IT 90
124 Uma 19 CSE 86
126 Kunti 21 IT 94
128 Draupadi 19 ECE 82
130 Lakshmi 21 CSE 93
132 Kausalya 19 ECE 89
133 Prithvi 22 CSE 81
134 Sarala 21 IT 85
136 Janaki 19 CSE 97
138 Urmila 21 IT 88
140 Mandakini 19 ECE 92
142 Shakuntala 21 CSE 84
144 Kamalini 19 ECE 91
145 Ranjit 22 CSE 87
147 Chaturbhuj 20 ECE 86
149 Omkar 22 CSE 90
```

Marks less than 80:

```
db.students.find({ marks: { $lt:
80 } })
```

Output:

```
102 Saraswati 21 IT 78
105 Hari 22 ME 74 107 Mohan 20 IT 69 110 Yamuna
21 IT 76
111 Krishna 20 ME 72
113 Rajendra 22 ECE 67
115 Gopal 20 CSE 79
117 Bhairav 22 ME 73
119 Indra 20 IT 68
121 Manohar 22 CSE 75
123 Vasudev 20 ME 71
127 Bhupendra 20 CSE 77
129 Ramesh 22 ME 65
131 Suresh 20 IT 70
135 Mahesh 20 ME 78
137 Narayan 22 ECE 74
139 Devendra 20 CSE 66
141 Ishwar 22 ME 69
143 Trilok 20 IT 79
146 Vidyawati 21 IT 73
148 Sushila 19 ME 68
150 Tulsi 21 IT 76
```

Marks equal to 85:

```
db.students.find({ marks: { $eq:
85 } })
```

Output:

```
134 Sarala 21 IT 85
```

Marks greater than or equal to 80:

```
db.students.find({ marks: { $gte: 80 } })
```

Output:

```
101 Vishnu 22 CSE 90
103 Govind 20 CSE 92
104 Parvati 19 ECE 81
106 Radha 21 CSE 88 108 Kamala 19 ECE 95
109 Shankar 22 CSE 83
112 Sita 19 CSE 89
114 Kaveri 21 IT 91
116 Savitri 19 ECE 84
118 Rukmini 21 CSE 96
120 Chandra 19 ECE 87
122 Gauri 21 IT 90
124 Uma 19 CSE 86
125 Dwarka 22 ECE 80
126 Kunti 21 IT 94
128 Draupadi 19 ECE 82
130 Lakshmi 21 CSE 93
132 Kausalya 19 ECE 89
133 Prithvi 22 CSE 81
134 Sarala 21 IT 85
136 Janaki 19 CSE 97
138 Urmila 21 IT 88
140 Mandakini 19 ECE 92
142 Shakuntala 21 CSE 84
144 Kamalini 19 ECE 91
145 Ranjit 22 CSE 87
147 Chaturbhuj 20 ECE 86
149 Omkar 22 CSE 90
```

Marks not equal to 85:

```
db.students.find({ marks: { $ne:
85 } })
```

Marks greater than 70 AND age less than 23:

```
db.students.find({ $and: [
{ marks: { $gt: 70 } },
{ age: { $lt: 23 } }
]
})
```

Output:

```
101 Vishnu 22 CSE 90
102 Saraswati 21 IT 78
103 Govind 20 CSE 92
104 Parvati 19 ECE 81
105 Hari 22 ME 74
106 Radha 21 CSE 88 108 Kamala 19 ECE 95
109 Shankar 22 CSE 83
110 Yamuna 21 IT 76
111 Krishna 20 ME 72
112 Sita 19 CSE 89
114 Kaveri 21 IT 91
115 Gopal 20 CSE 79
116 Savitri 19 ECE 84
117 Bhairav 22 ME 73
118 Rukmini 21 CSE 96
120 Chandra 19 ECE 87
121 Manohar 22 CSE 75
```

```
122 Gauri 21 IT 90
123 Vasudev 20 ME 71
124 Uma 19 CSE 86
125 Dwarka 22 ECE 80
126 Kunti 21 IT 94
127 Bhupendra 20 CSE 77
128 Draupadi 19 ECE 82
130 Lakshmi 21 CSE 93
132 Kausalya 19 ECE 89
133 Prithvi 22 CSE 81
134 Sarala 21 IT 85
135 Mahesh 20 ME 78
136 Janaki 19 CSE 97
137 Narayan 22 ECE 74
138 Urmila 21 IT 88
140 Mandakini 19 ECE 92
142 Shakuntala 21 CSE 84
143 Trilok 20 IT 79
144 Kamalini 19 ECE 91
145 Ranjit 22 CSE 87
146 Vidyawati 21 IT 73
147 Chaturbhuj 20 ECE 86
149 Omkar 22 CSE 90
150 Tulsi 21 IT 76
```

Department is CSE OR ECE:

```
db.students.find({ $or: [
{ department: "CSE" },
{ department: "ECE" }
]
})
```

Output:

CSE and ECE department students are returned.

CSE:

```
101 Vishnu 103 Govind 106 Radha 109 Shankar
112 Sita 115 Gopal 118 Rukmini 121 Manohar
124 Uma 127 Bhupendra 130 Lakshmi 133 Prithvi
136 Janaki 139 Devendra 142 Shakuntala 145 Ranjit
149 Omkar
```

ECE:

```
104 Parvati 108 Kamala 113 Rajendra 116 Savitri
120 Chandra 125 Dwarka 128 Draupadi 132 Kausalya
137 Narayan 140 Mandakini 144 Kamalini 147
Chaturbhuj
```

Marks NOT greater than 80:

```
db.students.find({ marks: { $not: { $gt:
80 } } })
```

Output:

```
102 Saraswati 21 IT 78
105 Hari 22 ME 74
107 Mohan 20 IT 69
110 Yamuna 21 IT 76
111 Krishna 20 ME 72
113 Rajendra 22 ECE 67
115 Gopal 20 CSE 79
117 Bhairav 22 ME 73
```

```
119 Indra 20 IT 68
121 Manohar 22 CSE 75
123 Vasudev 20 ME 71
125 Dwarka 22 ECE 80
127 Bhupendra 20 CSE 77
129 Ramesh 22 ME 65
131 Suresh 20 IT 70
135 Mahesh 20 ME 78
137 Narayan 22 ECE 74
139 Devendra 20 CSE 66
141 Ishwar 22 ME 69
143 Trilok 20 IT 79
146 Vidyawati 21 IT 73
148 Sushila 19 ME 68
150 Tulsi 21 IT 76
```

8. MongoDB Operator Summary

Operator	Meaning	Example
$gt	Greater than	marks: { $gt: 80 }
$lt	Less than	marks: { $lt: 80 }
$eq	Equal to	marks: { $eq: 85 }
$gte	Greater than or equal to	marks: { $gte: 80 }
$ne	Not equal to	marks: { $ne: 85 }
$and	All conditions must be true	marks > 70 AND age < 23
$or	At least one condition must be true	CSE OR ECE
$not	Negates a condition	NOT (marks > 80)

Total dataset: 51 students (Roll No. 101-151).

Implementation Details

- The project is implemented using MongoDB and executed through the MongoDB Shell (mongosh).
- A single collection named "students" is used, with fields: rollNo, name, age, department and marks.
- insertOne() and insertMany() are used to populate the collection with 51 student records.
- updateOne() and updateMany() with the $set operator are used to modify existing fields and addnew ones (e.g. status).
- deleteOne() and deleteMany() are used to remove individual and bulk records based on a filtercondition.
- find() is combined with comparison operators ($gt, $lt, $eq, $gte, $ne) and logical operators ($and,$or, $not) to retrieve specific subsets of student data.
- Each operation's acknowledgement object (acknowledged, matchedCount, modifiedCount,deletedCount) is used to verify that it executed as intended.

Results

All CRUD operations executed successfully on the students collection. The insertOne and insertMany operations populated the collection with 51 student records. The updateOne operation successfully changed one student's marks, while updateMany correctly modified all 17 CSE-department documents to add an "Active" status. The deleteOne operation removed one record and deleteMany found no records matching its filter, correctly reporting a deletedCount of 0. The query operations correctly filtered students by marks and department using comparison and logical operators, demonstrating that MongoDB's query engine returns accurate, condition-based results across a realistic-sized dataset.

Conclusion

This project highlights the practical use of MongoDB as a NoSQL database for managing structured academic data. By implementing insertion, update, deletion and a wide variety of query operations on a students collection, the project demonstrates a solid, hands-on understanding of MongoDB's Create, Read, Update and Delete (CRUD) capabilities as well as its comparison and logical query operators. The approach shows how a document database can be used to store and efficiently retrieve real-world tabular data such as student records.

Future Scope

- Use the MongoDB Aggregation Framework to compute department-wise averages, toppers andrankings.
- Add schema validation rules to enforce data types and required fields at the database level.
- Create indexes on frequently queried fields (e.g. marks, department) to improve performance.
- Build a front-end / web dashboard (e.g. using Node.js and Express) to interact with the databasevisually.
- Extend the collection with additional entities such as courses, attendance and results, and modelrelationships between them.

References

- MongoDB Official Documentation: https://www.mongodb.com/docs/
- MongoDB CRUD Operations: https://www.mongodb.com/docs/manual/crud/
- MongoDB Query Operators: https://www.mongodb.com/docs/manual/reference/operator/query/
- mongosh (MongoDB Shell) Documentation: https://www.mongodb.com/docs/mongodb-shell/

THANK YOU!